

ניתוח ועיצוב מערכות מידע

Use Case — תרשים תרחישי שימוש

הרצאה 4-5

עודכן: 25.08.2026

1 יסודות UML

2 תרחישי שימוש

3 תיעוד

4 קשרים

5 מניתוח לקוד

6 תרגול וסיכום

מאיפה הגענו: מהדרישות למודל

Previously on SAD — דרישות (הרצאה 3)

לאן ממשיכים היום

- מטקסט חופשי למודל חזותי
- תרשים תרחישי שימוש ([Use Case](#)ⁱ)
- מה המערכת עושה — ולמי
- גבול: מה בפנים ומה בחוץ

מה כבר יש לנו (הרצאה 3)

- זיהוי צרכים ([Wetherbe](#)ⁱ, [JAD](#)ⁱ, אבטיפוס)
- ניסוח דרישות [SMART](#)ⁱ
- [User Story](#)ⁱ מול דרישה קלאסית
- דרישת [CRUD](#)ⁱ

הדרישות בטקסט הן הקלט. היום נהפוך אותן למודל שאפשר לתקשר לאמת — ולתת ל-AI.

Joint Application = JAD · יצירה, קריאה, עדכון, מחיקה · Create, Read, Update, Delete = CRUD · עקיב · Specific, Measurable, Achievable, Relevant, Traceable = SMART · ספציפי, מדיד, בר-השגה, רלוונטי, עקיב · פיתוח יישומים משותף · Artificial Intelligence = AI · בינה מלאכותית · Development

SDLC – שלבים ובעלי תפקידים

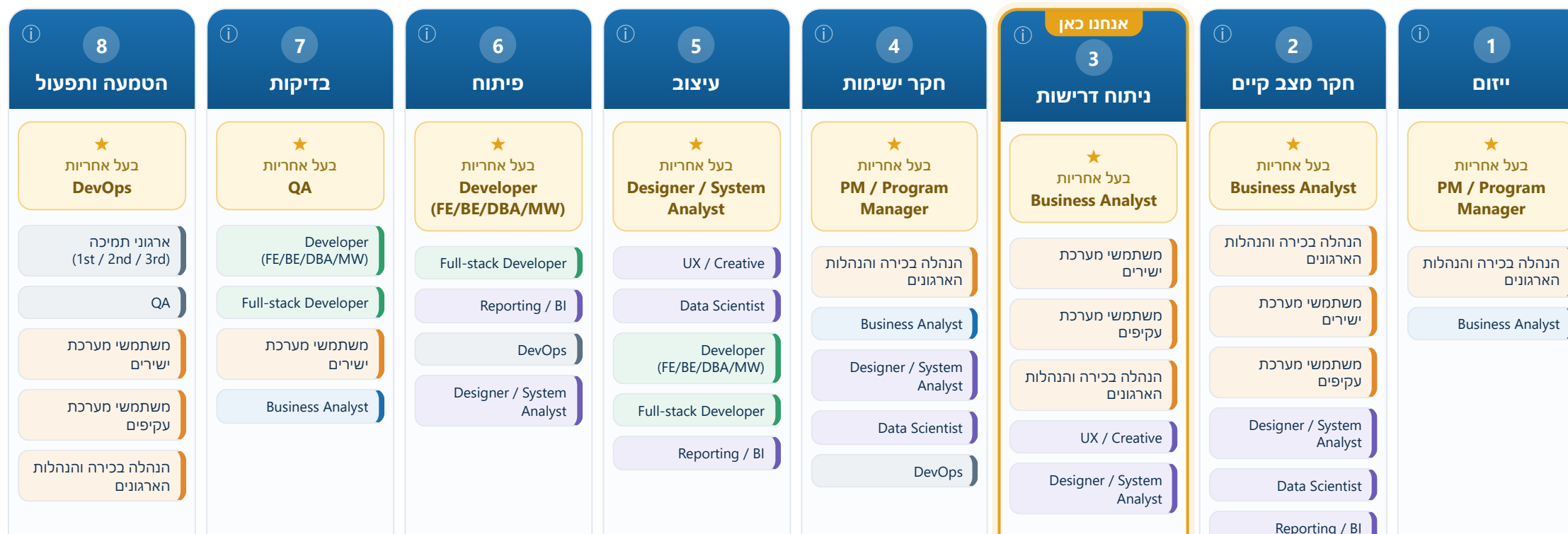
מי הבעלים של הדרישות, ולכן נכנס תרשים תרחישי השימוש

שלבי ה-SDLC – מי מוביל ומי משתתף בכל שלב

מיפוי בעלי התפקידים והאקטורים לאורך מחזור החיים של מערכת המידע

← הזמן מתקדם מימין לשמאל

לחצו על שלב או על תפקיד לקבלת הסבר קצר.



ניתוח דרישות

הסצנה: מערכת הזמנת אוכל בקמפוס

הקלט הגולמי — ראיונות עם בעלי עניין

נלווה מערכת אחת לאורך כל ההרצאה: מראיונות, דרך תרשים תרחישי שימוש (use case), ועד קוד רץ בהרצאה 13.

מנהל/ת קפיטריה 🍴

"בצהריים אנחנו קורסים. רוצה לראות הכל בלוח אחד, לעדכן מה אזל, ובסוף היום מה נמכר. מי שמזמין מראש ולא בא — האוכל לפח."

סטודנט/ית 🎓

"רבע שעה בין הרצאות, אין זמן לתור. רוצה להזמין מהטלפון, לשלם, ולדעת מתי מוכן. וצמחוני — שיהיה ברור מה מתאים לי."

עובד/ת דלפק איסוף 📋

"צריך לוודא שזו ההזמנה שלו וששולמה. רוצה קוד או ברקוד. ואם משהו השתבש — אני נותן החזר."

עובד/ת מטבח 🍳

"אני רק צריך לדעת מה להכין ובאיזה סדר. מסמן 'מוכן' והסטודנט מקבל הודעה. תשלומים? לא התפקיד שלי."

⚠️ הקלט גולמי, מעורפל וסותר

צרכים מעורבבים עם פתרונות, דרישות לא-פונקציונליות חבויות בתוך הטקסט, ובעיקר — סתירות בין בעלי העניין (למשל תשלום: כרטיס סטודנט מול אשראי מול מזומן). זה החומר שנברור ממנו.

דרישות פונקציונליות — רשימה למעקב

דרישה טובה היא SMART^① · לכל דרישה מזהה (FR) ועדיפות לפי MoSCoW^① — מה חובה בגרסה הראשונה ומה יכול לחכות

תזכורת: דרישה פונקציונלית מתארת פעולה/שירות שאפשר לבדוק שקרה — לא "המערכת תהיה נוחה", אלא מה בדיוק היא עושה.

מזהה · עדיפות	הדרישה (SMART)	מקור
FR-1 Should	המערכת תאפשר לסטודנט לסנן את התפריט לפי פרופיל תזונתי (צמחוני / טבעוני / ללא-גלוטן) ולהציג רק פריטים תואמים.	סטודנט
FR-2 Must	המערכת תשלח התראת "מוכן לאיסוף" תוך 10 שניות מרגע שהמטבח סימן את ההזמנה כמוכנה.	סטודנט
FR-3 Must	המערכת תאמת כל איסוף באמצעות קוד חד-פעמי בן 6 ספרות לפני מסירת ההזמנה.	דלפק
FR-4 Must	המערכת תאפשר למנהל לסמן פריט כ"אזל", ותסיר אותו מיידית מהתפריט הזמין להזמנה חדשה.	מנהל
FR-5 Won't	בסוף כל יום המערכת תפיק דוח מכירות המפרט כמות ומחזור לכל פריט, לפי קפיטריה.	מנהל
FR-6 Must	המערכת תאפשר למנהל לנהל את עובדי הקמפוס, תפקידיהם והרשאות הגישה שלהם.	מנהל
FR-7 Could	המערכת תאפשר לסטודנט לשמור הזמנה כמועדפת ולשחזר אותה להזמנה חדשה בלחיצה אחת.	סטודנט

Must = חובה (ליבת המערכת) · Should = רצוי · Could = אפשרי · Won't = לא בגרסה זו

המזהה והעדיפות מלווים כל דרישה — למעקב, בדיקה, ושיוך לתרחיש שמממש אותה.

Functional Requirement = FR · דרישה פונקציונלית · SMART = Specific, Measurable, Achievable, Relevant, Traceable · ספציפי, מדיד, בר-השגה, רלוונטי, עקיב

ניתוח דרישות

עד כמה מדויק — מספיק?

גם דרישה שנראית חדה מסתירה החלטה שטרם התקבלה
המנתח לא מנסח את הדרישה המושלמת בניסיון אחד — הוא יודע איזו שאלה חושפת את החסר. לכל דרישה מלמטה, השאלה שמחדדת אותה:

התראת "מוכן" תוך 10 שניות « **ומה אם ההתראה נכשלה — יש גיבוי? מה נחשב "נמסרה"?** reliability

קוד חד-פעמי בן 6 ספרות « **מאיפה הקוד, ומי מקליד — הסטודנט או הדלפק?** source / actor

להסיר פריט שאזל מהתפריט « **"הסרה" = נעלם לגמרי, או מוצג כ"לא זמין"?** semantics

דוח מכירות בסוף היום « **נוצר איפה, נשלח למי, ולמה לא לפי דרישה?** source / dest / timing

⚠ **הדרישה לעולם לא "גמורה"**
תמיד יש שאלה שמחדדת. אותו הרגל שאתם מפעילים על פלט של AI — הפעילו על הדרישה עצמה: מה עוד לא נאמר?

המיומנות של המנתח: לדעת איזו שאלה לשאול — לא לנסח מושלם מהניסיון הראשון.

אותן דרישות — הפעם עד הסוף

דרישה טובה סוגרת כל החלטה פתוחה — עד שאפשר לבדוק שהתקיימה

- ✓ מפורטת
- ✓ חד-משמעית
- ✓ ניתנת לבדיקה
- ✓ עקיבה (מזהה)

מה הוכרע	הדרישה — מלאה ומדויקת	מזהה · עדיפות
שילוב פרופילים + שמירה	המערכת תאפשר לסטודנט לבחור פרופיל תזונתי אחד או יותר (צמחוני / טבעוני / ללא-גלוטן); התפריט יציג רק פריטים שסומנו כתואמים לכל הפרופילים שנבחרו, והבחירה תישמר לפרופיל להזמנות הבאות.	FR-1 Should
מסירה מובטחת + גיבוי	עם סימון ההזמנה כ"מוכנה", המערכת תודיע לסטודנט תוך 10 שניות, ותוודא שההודעה נמסרה — עם ניסיון גיבוי אם המסירה הראשונה נכשלה.	FR-2 Must
מקור הקוד + מי מקליד	בעת אישור ההזמנה המערכת תפיק קוד איסוף בן 6 ספרות ותציג אותו לסטודנט באפליקציה ; עובד הדלפק יקליד את הקוד שהסטודנט מציג, וההזמנה תשוחרר רק בהתאמה.	FR-3 Must
משמעות "הסרה"	בסימון פריט כ"אזל", המערכת תציג אותו בתפריט כ" לא זמין " (מעומעם, ללא הוספה לעגלה) ולא תמחק אותו; הפריט יחזור להיות זמין בביטול הסימון.	FR-4 Must
תזמון + יעד הדוח	בסוף כל יום עסקים (23:00) המערכת תפיק אוטומטית דוח מכירות לכל קפיטריה — כמות ומחזור לכל פריט — ותשלח אותו בדוא"ל למנהל ; הדוח יהיה זמין גם להפקה לפי דרישה.	FR-5 Won't

אבל דרישה מפורטת עדיין אוגדת כמה התנהגויות — כדי שכל אחת תהיה אטומית וניתנת לבדיקה, מפרקים אותה לתת-דרישות (מיד).

● ניתוח דרישות

דרישה כללית ← דרישות מפורטות

דוגמה — ניהול זמינות פריטים (FR-4 מהרשימה, עכשיו מפורטת)

Must

שחקן: מנהל

דרישה כללית FR-4

המערכת תאפשר למנהל לנהל את זמינות פריטי התפריט.

הדרישות המפורטות (SMART, ניתנות לבדיקה):

FR-4.1 המערכת תאפשר למנהל לסמן פריט תפריט כ"אזל" או להחזירו למצב "זמין".

FR-4.2 המערכת תציג פריט "אזל" בתפריט כ"לא זמין" (מעומעם, ללא אפשרות הוספה לעגלה), ולא תמחק אותו.

FR-4.3 המערכת תשקף שינוי זמינות לסטודנטים תוך 5 שניות מרגע העדכון.

FR-4.4 המערכת תאפשר שינוי זמינות של פריט רק למשתמש בהרשאת "מנהל".

הכללית אומרת מה מתאפשר; המפורטות מגדירות את ההתנהגות המדויקת.

Functional Requirement = FR · דרישה פונקציונלית · Specific, Measurable, Achievable, Relevant, Traceable = SMART · ספציפי, מדיד, בר-השגה, רלוונטי, עקיב

דרישה כללית ← דרישות מפורטות

דוגמה — אימות ומסירת הזמנה (FR-3 מהרשימה, עכשיו מפורטת)

FR-3 דרישה כללית שחקן: עובד דלפק Must

המערכת תאפשר לעובד הדלפק לאמת ולמסור הזמנה באמצעות קוד איסוף חד-פעמי.

הדרישות המפורטות (SMART, ניתנות לבדיקה):

FR-3.1 עם אישור ההזמנה, המערכת תפיק קוד איסוף חד-פעמי בן 6 ספרות ותציג אותו לסטודנט באפליקציה.

FR-3.2 המערכת תשחרר הזמנה למסירה רק לאחר שעובד הדלפק הקליד קוד התואם במדויק לקוד ההזמנה.

FR-3.3 אם הקוד שגוי, המערכת לא תשחרר את ההזמנה ותציג לעובד הדלפק הודעת שגיאה.

הכללית אומרת מה מתאפשר; המפורטות הן שלבי התהליך (כולל חריג).

Functional Requirement = FR · דרישה פונקציונלית · Specific, Measurable, Achievable, Relevant, Traceable = SMART · ספציפי, מדיד, בר-השגה, רלוונטי, עקיב

מה, לא איך — דרישה מתארת צורך, לא פתרון

המנתח מגדיר מה המערכת צריכה להשיג · איך לממש — החלטה של שלב העיצוב

מבחן מהיר: אם אפשר לדמיין שני מימושים שונים שמקיימים את המשפט — זו דרישה (מה). אם המשפט נועל מנגנון יחיד — נכנס אליו "איך".

מזהה	הניסוח עם "איך" (הפתרון כבר בפנים)	הניסוח העדיף — מה בלבד	ה"איך" שייך ל-
FR-4.2	"...מוצג כ'לא זמין' (מעומעם , ללא הוספה לעגלה)"	"...מוצג כ'לא זמין' ולא ניתן להזמנה"	עיצוב / UI
FR-3.1	"...תפיק קוד חד-פעמי בן 6 ספרות שהדלפק מקליד"	"המערכת תוודא שההזמנה נמסרת למי שהזמין"	מנגנון אימות
FR-2	"...ההתראה תישלח ב- SMS ו-Push "	"המערכת תודיע לסטודנט באופן אמין ומיידי"	ערוץ תקשורת

ה"איך" לא שגוי — רק מוקדם: הוא שייך לעיצוב, ומשאיר למתכנן חופש לבחור. הדרישה נשארת יציבה גם כשהמימוש מתחלף.

דרישת CRUDⁱ קלאסית — ניהול עובדים והרשאות

יכולת חדשה בקמפוס: מי מורשה להיכנס למערכת ובאיזה תפקיד · פעולת "ניהול" = ארבע תת-דרישות, והשדות מוגדרים פעם אחת

— יכולת שאינה קיימת היום

המערכת תאפשר למנהל לנהל את עובדי הקמפוס, תפקידיהם והרשאות הגישה שלהם.

שחקן: מנהל

Must FR-6

שדות העובד — מוגדרים פעם אחת (מילון נתונים):

טלפון · רשות

סטטוס · פעיל / לא-פעיל

קפיטריה

דוא"ל · פורמט תקין

תפקיד · דלפק / מטבח / מנהל

שם משתמש · ייחודי

שם מלא

כל פעולה מפנה ל"שדות העובד" — לא חוזרת על הרשימה. חובה/רשות, ייחודיות ופורמט מוגדרים כאן, פעם אחת.

הפעולות — כל תת-דרישה נכתבת כ"המערכת ת..." (The system shall) — פעולה שאפשר לבדוק:

Read · קריאה FR-6.2

R

6.2.1 המערכת תאפשר למנהל לצפות בפרטי עובד.

6.2.2 המערכת תאפשר לחפש ולסנן עובדים לפי תפקיד, קפיטריה או סטטוס.

6.2.3 המערכת תציג לעובד את הפרופיל שלו בלבד.

Create · יצירה FR-6.1

C

6.1.1 המערכת תאפשר למנהל להוסיף עובד עם שדות החובה ולשייך לו תפקיד.

6.1.2 המערכת תקצה מזהה ייחודי ותדחה שם-משתמש שאינו ייחודי.

6.1.3 אם שדה חובה חסר/שגוי — המערכת לא תיצור את הרשומה ותציג שגיאה.

Delete · מחיקה FR-6.4

D

המערכת תסיר רשומת עובד לצמיתות רק בתנאים מוגדרים (טעות / התיישנות / חוק). ← מחיקה אינה פעולה אחת: בשקף הבא.

Update · עדכון FR-6.3

U

המערכת תאפשר למנהל לעדכן את פרטי העובד, תפקידו והרשאותיו (למעט המזהה). במעבר ל"לא-פעיל" המערכת תחסום גישה ותשמור את הרשומה (מחיקה רכה).

השדות מוגדרים פעם אחת · כל פעולה מפורקת לעומק שהיא צריכה — יצירה וקריאה דורשות כמה תת-דרישות, עדכון ומחיקה שורה אחת. לא תמיד שורה אחת.

ניתוח דרישות

Functional Requirement = FR · דרישה פונקציונלית · Create, Read, Update, Delete = CRUD · יצירה, קריאה, עדכון, מחיקה

מחיקה — לא פעולה אחת

רוב ה"מחיקות" שאנחנו מכירים הן בעצם עדכון סטטוס · מחיקה אמיתית נדירה — ומוסדרת

מחיקה רכה **סימון "לא-פעיל" = Update**

שינוי סטטוס בלבד — הרשומה נשארת, הגישה נחסמת, וההיסטוריה נשמרת. זה מה שהמערכת צריכה ביום-יום (עובד עוזב → "לא-פעיל"). זו לא מחיקה.

מחיקה אמיתית · Delete **הסרת הנתון לצמיתות — מותרת רק כאשר:**



דרישה חוקית

נושא-המידע ביקש למחוק את המידע האישי עליו.



התיישנות

עברה תקופת השמירה שמוגדרת במדיניות/בחוק.



טעות

הרשומה נוצרה בטעות ויש להסירה.

💡 לדיון: האם עובד (או סטודנט) יכול לדרוש שנמחק את כל המידע עליו? — "הזכות להישכח" (GDPR). מה זה מחייב בדרישות המערכת?

ברירת המחדל ברוב המערכות: מחיקה רכה. מחיקה אמיתית דורשת הרשאה, תיעוד (audit), ולעיתים מחיקה מדורגת (cascade).

User Stories — אותן דרישות, מבט המשתמש

אותה דרישה, מנקודת מבט המשתמש (User Story^①) והערך שהוא מקבל

כ[תפקיד], אני רוצה [יכולת], כדי [ערך]

- כסטודנט/ית, אני רוצה להזמין ולשלם מהטלפון מראש, כדי לאסוף בלי להמתין בתור בהפסקה של 15 דקות.
- כסטודנט/ית, אני רוצה לראות רק מנות התואמות לפרופיל התזונתי שלי, כדי לבחור מהר ובבטחה.
- כמנהל/ת קפיטריה, אני רוצה לראות את כל ההזמנות הפעילות בלוח אחד, כדי לנהל את עומס הצהריים.
- כעובד/ת מטבח, אני רוצה לראות תור הכנה לפי עמדה (חם / קר), כדי להכין בסדר הנכון.
- כעובד/ת דלפק, אני רוצה לאמת הזמנה בקוד ייחודי, כדי למסור לסטודנט הנכון ולמנוע טעויות.

⚠ **המבחן של User Story טוב**

לכל אחת יש ערך אמיתי ב"כדי" — לא "כדי שהמערכת תעבוד". אם אין ערך למשתמש, זו לא דרישה אמיתית.

דרישות לא-פונקציונליות (NFR) — כמה טוב

איכות ואילוץ שחלים על המערכת כולה — לא פעולה בודדת

איך מבחינים? אותה מילה — קנה מידה אחר

✓ פונקציונלי

"שם פרטי ומשפחה של סטודנט יכולים להיות בעברית או באנגלית" — התנהגות של שדה מסוים.

◆ לא-פונקציונלי

"כל קלט במערכת יכול להיות בעברית או באנגלית" — איכות רוחבית על כל המערכת.

דוגמאות מהמערכת שלנו

- ביצועים: תהליך הזמנה מלא (מבחירה ועד אישור תשלום) יושלם תוך פחות מ-90 שניות.
- עומס שיא: תמיכה ב-500 הזמנות במקביל בין 12:00-14:00, ללא זמן תגובה מעל 2 שניות.
- אבטחה ופרטיות: אימות דרך מערכת הזיהוי של האוניברסיטה בלבד; לא יאוחסנו סיסמאות או פרטי אשראי.
- זמינות: זמינות של 99.5% בשעות הפעילות (08:00-20:00).

⚠ ה-NFR אינן תרחישי שימוש

תרשים ה-use case מתאר פונקציונליות בלבד. את הלא-פונקציונליות מתעדים לצידו כאילוץ — הן משפיעות על כל תרחיש, אך אינן תרחיש בפני עצמן.

פונקציונלי ← תרחיש בתרשים · לא-פונקציונלי ← אילוץ שמתועד לצד התרשים.

שפה חזותית סטנדרטית (תקן OMG) לתיאור, מידול ותיעוד של מערכות תוכנה - מנותקת משפת תכנות מסוימת.

מה זה

- שפה אחת משותפת לאנליסטים, מפתחים ובעלי עניין
- נוטציה, לא מתודולוגיה - לא אומרת לך איך לנהל פרויקט
- נולדה מתכנות מונחה-עצמים (OOD)

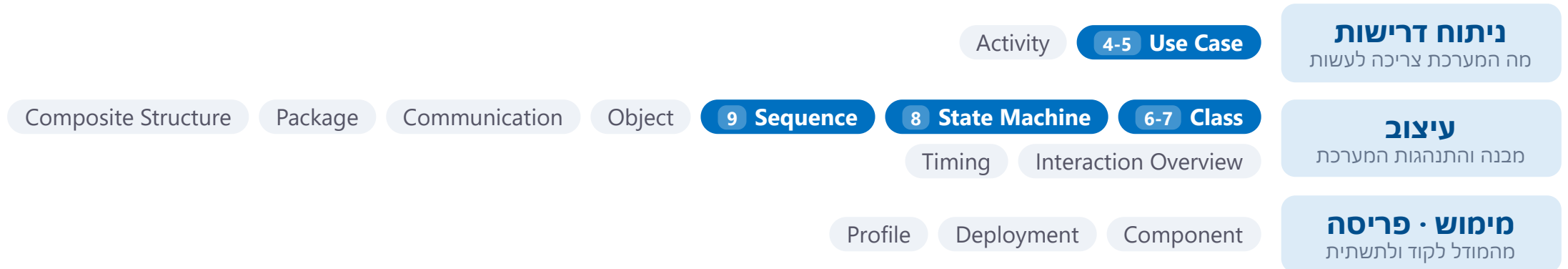
למה UML?

- מסמכת החלטות ומאפשרת לאתגר אותן
- מנותקת משפת תכנות - לא ננעלת על מימוש מסוים
- בהירות משותפת על שחקנים, גבולות והתנהגות

UML היא נוטציה משותפת - והמודל הוא המפרט שמתעד החלטות ומאפשר לאתגר אותן.

UML לאורך מחזור חיי הפיתוח

כ-14 סוגי תרשימים; הקורס מתמקד ב-4 שנותנים את התמונה המלאה



מה נלמד — הציר המרכזי

הכוחלים = נלמדים בקורס (עם מספר ההרצאה). ארבעה תרשימים לתמונה מלאה: Use Case (דרישות) ← Class (מבנה) ← State + Sequence (התנהגות).

UML מרוכז בדרישות ובעיצוב — לא לאורך כל ה-SDLC.

SDLC = Software Development Life Cycle · מחזור פיתוח תוכנה · UML = Unified Modeling Language · שפת מידול אחידה

ההיסטוריה של UML

שלושת ה'אמיגוס' שאיחדו את השפה

Ivar Jacobson

OOSE

מיקוד ב-Use Cases

Jim Rumbaugh

OMT

רמה גבוהה, מיקוד בניתוח

Grady Booch

OOAD

רמה נמוכה, מיקוד בעיצוב

איחוד תחת ה-OMG

שלוש השיטות אוחדו ל-UML (תקן OMG, אמצע שנות ה-90). התרומה של Jacobson — תרשימי Use Case — היא בדיוק הנושא שלנו היום.

למה ללמוד UML? — כי השוק מחפש את זה

כמעט אף פעם לא שם המשרה — אבל כישור נדרש או מועדף בתפקידים מבוקשים

חפשו "UML" בלוחות הדרושים והוא חוזר שוב ושוב — כמעט תמיד לא כשם התפקיד, אלא כדרישה או יתרון בתיאור המשרה. Use Case הוא נקודת הכניסה, ושליטה ב-UML פותחת דלת לתפקידים כמו:

מוצר 📦

Product Owner

Product Manager

ניתוח 🔍

Requirements Engineer

Systems Analyst

Business Analyst

בדיקות ואיכות ✅

QA Automation Engineer

QA / Test Analyst

עיצוב וארכיטקטורה 🏠

Software Designer

System Architect

Solution Architect

ובעידן ה-AI 🤖

ה-AI ינסח טיוטת תרשים מהתיאור שלכם — אבל אתם מגדירים מה המערכת עושה ומה בגבול. ככל שתנסחו טוב יותר, כך הטיוטה טובה יותר — וגם תזהו מתי היא שגויה. זה מה שמבדיל אנליסט/ית מעולה מבינוני/ת.

חפשו בעצמכם 🔗

[UML בלינקדאין \(ישראל\)](#) · [Indeed ב-UML](#)

האם UML עדיין רלוונטי?

הפרובוקציה של 2026

**“ה-AI כותב את הקוד מהפרומפט.
בשביל מה לבזבז זמן על תרשימים?”**

הפיתוי לדלג אמיתי:

“כבר נתקן תוך כדי”

לחץ זמן ודדליינים

AI מייצר קוד ותרשימים בשניות —
אז למה לטרוח?

אבל זו טעות — ובשקף הבא נראה בדיוק למה.

ניתוח דרישות

UML = Unified Modeling Language · שפת מידול אחידה · AI = Artificial Intelligence · בינה מלאכותית

מודל הוא חשיבה, לא ציור

למה UML רלוונטי — ודווקא בעידן ה-AI

- מודל הוא חשיבה, לא ציור — התוצר הוא בהירות על שחקנים, גבולות והתנהגות
- המודל הוא המפרט (spec) שה-AI צריך — בינה מגבירה את מה שנתת לה, לטוב ולרע
- גבול המערכת וההיקף הם החלטה עסקית — לא משהו שבינה מחליטה במקומך
- המודל הוא משטח האימות — אי אפשר לאתגר קוד שלא הבנת

הסמנטיקה חשובה עכשיו יותר מתמיד

בעבר מתכנת קרא את המודל והשלים פערים בשכל ובשאלת הבהרה. היום מכונה קוראת אותו ומשחזרת בדיוק את מה שכתבת, כולל העמימות והטעויות. אי-דיוק כבר לא עולה שיחה; הוא עולה קוד שגוי, שנכתב בביטחון.

עלות הציור צנחה לאפס. ערך האנליסט עובר מ'לצייר' ל'להחליט ולאמת'.

UML = Unified Modeling Language · שפת מידול אחידה · AI = Artificial Intelligence · בינה מלאכותית

ולמה בכלל למדל — וגם מה מודל אינו

הגדרה

Dr. James Rumbaugh — "Modeling captures essential parts of the system"

למה למדל?

- הפשטה — מתמקדים בחיוני, מתעלמים מהרעש
- מאפשר לחקור תכונות ולחזות התנהגות לפני שבונים
- שפת תקשורת בין כל בעלי העניין
- ממפה מצב קיים (As-Is) מול מצב רצוי (To-Be)

⚠ **אבל — המודל אינו המערכת ("Bubbles don't crash", Meyer)**

ה'בועות' בתרשים לא מתקמפלות ולא רצות — מודל יפה יכול להיראות נכון ולהיות שגוי. בעידן ה-AI נוצרים מודלים משכנעים-אך-שגויים בזול, ולכן חובה לאמת את המודל מול המציאות.

משפחות תרשימי UML - מפת הקורס

איפה כל תרשים יושב - ומתי נלמד אותו

Management (ניהולי)

תיעוד ותכנון אספקה — מעבר לתרשימים שנלמד
■ חבילות, פריסה ורכיבים

Dynamic (התנהגותי)

איך המערכת מתנהגת לאורך זמן
■ State Diagram
■ Sequence Diagram

Structural (מבני)

מה קיים במערכת — מבנה ודרישות
■ Use Case **היום**
■ Class Diagram

תרשים	משפחה	מתי בקורס
Use Case עכשיו	Structural (מבני)	היום (4-5)
Class Diagram	Structural (מבני)	הרצאה 6-7
State Diagram	Dynamic (התנהגותי)	הרצאה 8
Sequence Diagram	Dynamic (התנהגותי)	הרצאה 9

איפה אנחנו עכשיו

Use Case = שכבת הדרישות. כל השאר נבנה מעליה.

מכאן — בונים את התרשים

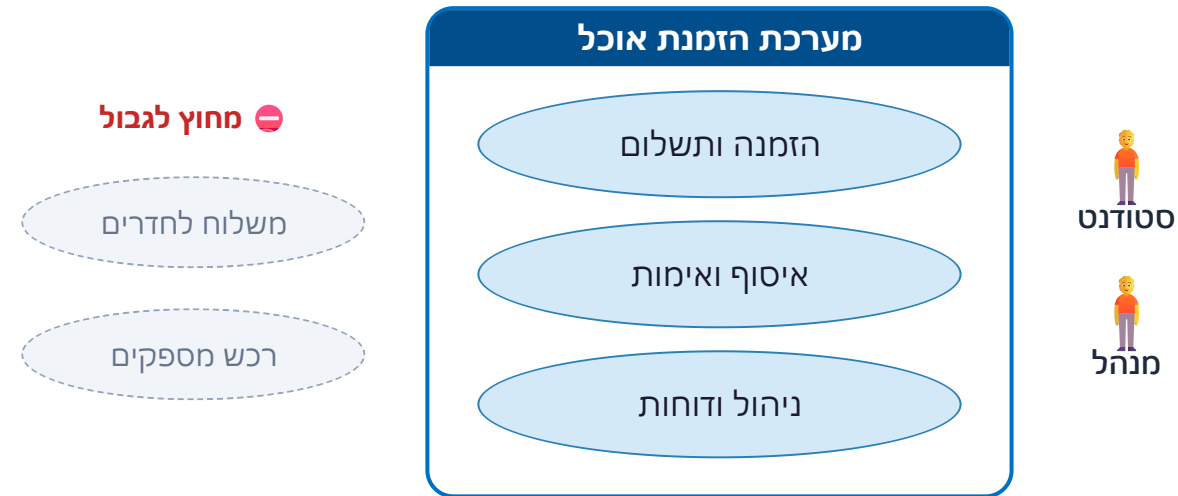
תרשים תרחישי שימוש

Use Case Diagram



גבול המערכת (System Boundary)

התיבה: תרחישים בפנים, שחקנים בחוץ — ומה נכנס זו החלטה עסקית



⚠ הבינה ממציאה פיצ'רים שמחוץ לגבול

AI מוסיף "בעצמו" יכולות מפתות שלא ביקשתם — משלוח, רכש מספקים — כולן ב-OUT. כש-AI ממציא משלוח, הוא חצה גבול שאתם הגדרתם. הגבול הוא שלכם.

בתוך התיבה = תרחישי השימוש · מחוצה לה = השחקנים. מה נכנס — החלטה אנושית, לא של ה-AI.

מערכת גדולה — כמה תרשימים, לכל אחד אזור משלו

כשהמערכת גדלה מפצלים לתת-מערכות — כל אחת תרשים use case נפרד עם גבול משלה
תרשימים אחד עם עשרות תרחישים הופך לבלתי-קריא. מפצלים לפי אזורי אחריות (subsystems). לדוגמה — מערכת
הזמנת האוכל שלנו מתחלקת לשלושה אזורים:



במערכות אמת 💡

בנק או בית-חולים = עשרות תרשימי use case. שחקן יכול להופיע בכמה אזורים. עיקרון ה-KISS (Keep It Simple, Stupid) שמור על פשטות): כל תרשימים ממוקד וקריא — לא הכול על דף אחד.

Actor - שחקן

מי שיוזם תרחיש במערכת — אדם, מערכת אחרת, או זמן

שחקן = תפקיד (לא אדם ספציפי) שיוזם תרחיש במערכת — הוא זה שמתחיל.

שחקנים לא-אנושיים

🕒 זמן / מתזמן

💳 שער תשלום — כשהוא מודיע לנו (החזר / כשל חיוב)

הזמן מפעיל תרחישים מתוזמנים (דוח סוף-יום) — ולכן הוא שחקן. מערכת חיצונית שמפעילה אותנו מיוזמתה — היא שחקן לכל דבר. אותה מערכת בכיוון ההפוך (אנחנו פונים אליה) היא קריאת API ולא שחקן.

שחקנים (אנושיים)

🎓 סטודנט/ית

🍽️ מנהל/ת קפיטריה

👷 עובד/ת מטבח · דלפק

⚠️ שניים שנראים כמו שחקן — ואינם

- ❑ עקיף — רק מקבל פלט או מושפע (הנהלה שמקבלת דוח מודפס)
- ❑ מה שאנחנו קוראים לו — 🔔 שירות התראות · 🏠 חשבונות אוניברסיטה, וגם 💳 שער תשלום כשאנחנו מחייבים דרכו: אנחנו פונים אליהם. זו קריאת API — הכיוון הוא שקובע.

✅ יוזם = שחקן

מי שמתחיל את התרחיש — סטודנט, מנהל, עובד דלפק... וגם 🕒 זמן.

מבחן השחקן: מי יוזם? הוא מפעיל אותנו — או שאנחנו מפעילים אותו?

שחקן או לא? — דיון בכיתה

המבחן: מי יוזם? הוא מפעיל אותנו — או שאנחנו מפעילים אותו?

1



סטודנט שמזמין אוכל
מהאפליקציה

2



שער התשלום שמאשר עסקה

3



הנהלה שמקבלת דוח חודשי
במייל

4



עובד מטבח שמסמן הזמנה
כ"מוכנה"

5



הזמן — מפעיל אוטומטית דוח
סוף-יום

6



טכנאי IT שמתחזק את השרת

7



חבר שאוכל מנה שהוזמנה עבורו

8



מדפסת שמדפיסה מדבקת
הזמנה

רמז: מי שמושפע, מי שרק מקבל פלט, ומה שאנחנו קוראים לו — אף אחד מהם אינו שחקן. רק מי שיוזם.

ניתוח דרישות

Use Case — הגדרה

Ivar Jacobson — מה המערכת עושה עבור שחקן

ההגדרה של Jacobson

תיאור של רצף פעולות שהמערכת מבצעת, המניב תוצאה בעלת ערך נצפה לשחקן מסוים.

מתאר מה צריך לקרות — לא איך 

מניב תוצר בעל ערך לשחקן 

מאותחל תמיד ע"י שחקן — הוא היוזם 

מחובר לשחקנים בקשרים 

מנקודת מבט השחקן — לא תהליך פנימי 

בגודל נכון — לא צעד טכני, לא תת-מערכת 

בדוגמה שלנו — שם = פועל + מושא

) הזמנת אוכל

) עיון בתפריט

) סימון הזמנה כמוכנה

תרחיש שימוש או לא? — דיון בכיתה

המבחן: יחידת ערך שלמה שמאותחלת ע"י שחקן? (לא צעד טכני, לא כל המערכת, לא דרישה לא-פונקציונלית)

1



הזמנת אוכל ותשלום

2



אימות איסוף בדלפק

3



התחברות למערכת (Login)

4



שלח מייל

5



ניהול הקפיטריה

6



זמן תגובה מתחת ל-2 שניות בעומס

ניתוח דרישות

רמז: לא כל דבר שקורה במערכת הוא תרחיש — רק יחידת ערך שלמה לשחקן.

התרשים כמכלול — Use Case Diagram

גבול + שחקנים + תרחישים + קשר בסיסי — הכול יחד, בדוגמה פשוטה



מה יש בתרשים?

- גבול — התיבה ✓
- שחקנים — בחוץ ✓
- תרחישים — בפנים ✓
- association — קו ✓
- שחקן ↔ תרחיש

לכל תרחיש יש פירוט 💡

התרשים הוא המפה — מאחורי כל תרחיש יש תיאור שלם. לחצו על תרחיש בתרשים.

בהמשך — חלק ג'

קשרים בין תרחישים: «include», «extend», והורשה.

מהראיון לתרחישים — 3 שלבים בהובלת האנליסט/ית

בכל שלב: האנליסט/ית העסקי/ת (Business Analyst) מחליט/ה ומאמת/ת · ה-AI מזרז

1 ראיונות

מנהל/ת ומחדד/ת את הראיונות —
עד שכל המידע הדרוש נאסף.
מסייע בתיעוד ובארגון הנתונים
שנאספו.

interview-notes.md

2 כתיבת דרישות

מוודא/ת שהדרישות נכונות ותואמות
למה שהלקוח רוצה וצריך.
מסייע לנסח ולסווג (FR / NFR).

requirements.md — טבלה ממוספרת (FR-)
(1...) למעקב ולשיוך לתרחיש

3 תרשים Use Case

מחליט/ה גבול, שחקנים, תרחישים
וקשרים — ומאמת/ת מול הדרישות.
מפיק טיוטת תרשים מהדרישות.

תרשים use case

האיכות בידי האנליסט/ית — ה-AI מזרז, לא מחליט.

Non-Functional Requirement = NFR · דרישה לא-פונקציונלית · Functional Requirement = FR · דרישה פונקציונלית · Artificial Intelligence = AI · בינה מלאכותית

למה לתעד מעבר לתרשים?

התרשים הוא המפה - לא הטיול

שתי רמות תיעוד:

תיאור מפורט — Flow of Events

כל הצעדים, החלופות והחריגים.
כאן חי הטיול עצמו — צעד אחר צעד, כולל מה שמשתבש.

תיאור ברמה גבוהה

פסקה קצרה: מה התרחיש עושה ולמי.
מספיק כדי לדעת שהתרחיש קיים ולמי הוא שייך.

 **התיעוד הוא הקלט**

התיעוד הוא מה שמאפשר לאמת - ומה ש-AI צריך כדי לבנות נכון.

• ניתוח דרישות

מה יש בתוך תרחיש מתועד?

התרשים אמר אילו תרחישים יש — התיעוד אומר מה קורה בכל אחד

זרימה אחת — ושתי הסתעפויות

ראשית (Main) — הזרימה
מסלול ההצלחה. רק היא עומדת בפני עצמה.

חלופית (Alternate) — מסתעפת
וריאציות תקינות — בתוך הזרימה, בנקודת ההחלטה.

חריג (Exception) — נתלה
כשמשהו נכשל — על הצעד שיכול להיכשל. לא רץ לבדו.

בספרות ובכלים תפגשו גם "זרימה חלופית" / "זרימת חריג" — אותם מושגים בדיוק.

שדות כותרת

שדה	תיאור
שם התרחיש	מזהה את ה-use case
שחקן	מי יוזם את התרחיש
תנאי קדם (Pre-condition)	מה חייב להתקיים לפני
תנאי סיום (Post-condition)	מה מובטח אחרי

המבנה הזה הוא בדיוק מה ש-AI יודע למלא - ומה שאתם יודעים לאתגר.

לכתוב Flow of Events — מה זה בעצם?

לא סיפור — רצף של צעדים שאפשר לבדוק

כל צעד:

מי עשה מה, ומה המערכת החזירה. פעולה אחת שניתן לראות — לא סיכום עסקי.

✗ צעד שמסתיר תת-תהליך

"הסטודנט משלם על ההזמנה" · "המערכת מטפלת בבקשה"

✓ אותו דבר, מפורק

1. סטודנט: מזין מספר כרטיס, תוקף ו-CVV
2. המערכת: שולחת בקשת חיוב לשער התשלומים
3. המערכת: מקבלת אישור ופותחת את ההזמנה

⚠ אף פעם לא "אם... אחרת" בתוך צעד

תנאי שנכתב כפרוזה נעלם מהקורא. הוא מבנה — ראו משמאל.

ארבעה מבנים — לכל אחד בית אחד

מה אני מתאר	המבנה
פעולה שחוזרת עד שנגמר	לולאה (loop) בזרימה הראשית
החלטה עסקית — כמה תוצאות תקינות	חלופית (alt) בזרימה הראשית
משהו נכשל — קלט שגוי, נדחה, לא נמצא	חריג (extension) תלוי בצעד שנכשל
פעולה אחרת או מסלול אחר לאותה מטרה	תרחיש נפרד (scenario)

בהמשך, אחד-אחד: תרחיש · חריג · תנאי קדם וסיום.

מה זה תרחיש (scenario) — ומה לא?

"ניהול X" עם תרחיש אחד מעורפל = התיעוד הכי רזה שיש

✓ שתי סיבות מוצדקות — ורק הן

✗ מה שנראה כמו תרחיש — ואינו

תוצאת שגיאה אחרת

"התשלום נדחה" אינו תרחיש — הוא חריג של תרחיש קיים.

בחירה עסקית בתוך מסלול אחד

"שילם בכרטיס סטודנט או באשראי" אינו תרחיש — הוא חלופית (alt) בתוך הזרימה.

⚠ מה AI נוטה לעשות

כותב ל-"ניהול תפריט" תרחיש אחד — "המנהל מנהל את התפריט" — ומדלג על שלוש פעולות.

1 · פעולה נפרדת בתוך CRUD^①

UC מסוג "ניהול X" חייב תרחיש לכל פעולה:

חוספה צפייה עדכון מחיקה

בדיוק ה-CRUD שכתבנו כדרישה — עכשיו בזרימה.

2 · אותה מטרה, מסלול אחר

נתיב או טריגר שונה מהותית — "רישום רגיל" מול "רישום מאוחר עם קנס".

תרחיש = מטרה שהושגה במסלול שלם. שגיאה ← חריג · בחירה ← חלופית.

ניתוח דרישות

CRUD = Create, Read, Update, Delete · יצירה, קריאה, עדכון, מחיקה · AI = Artificial Intelligence · בינה מלאכותית

כך נראה Flow of Events מלא

אותם ארבעה מבנים — בתוך תרחיש אמיתי. לחצו על תרחיש ברשימה.

מה לחפש

- 2 — חזור על הוספת פריטים
- 4 — לפי אמצעי התשלום
- 2.2a — חריג ↩ חוזר לשלב
- מספור: נקודה = זרימה · אות = חריג

שני תרחישים שונים

הזמנה ותשלום — תרחיש אחד עשיר.
ניהול תפריט ומלאי — ארבעה תרחישים, אחד לכל פעולת ^①CRUD.

זה אותו תיעוד

בדיוק מה שנפתח בלחיצה על תרחיש בתרשים — ומה שנראה עוד מעט ב-Visual Paradigm.

ניהול תפריט ומלאי

הזמנה ותשלום

הזמנה ותשלום שחקן: סטודנט

תנאי קדם

- הסטודנט מזוהה באפליקציה.
- לקפיתריה שנבחרה יש תפריט פעיל.

תנאי סיום

- ההזמנה נקלטה בסטטוס "שולמה" ושובצה לתור ההכנה.
- לסטודנט הוצגו מספר הזמנה וקוד איסוף.

תרחישים (1)

1 מסלול מרכזי

תרחיש אחד — אבל בתוכו לולאה, החלטה עסקית ושלושה חריגים.

מסלול מרכזי

1 סטודנט: בוחר קפיתריה ופותח את התפריט שלה

2 חזור הסטודנט מוסיף פריטים להזמנה:

2.1 סטודנט: בוחר פריט מהתפריט ומזין כמות

2.2 המערכת: מוסיפה את הפריט לעגלה ומחשבת סכום ביניים

2.2a אם הפריט אזל מהמלאי בזמן הרכבת ההזמנה:

2.2a1 המערכת מסמנת את הפריט "אזל" ומסירה אותו מהעגלה

2.2a2 המערכת מציעה פריטים חלופיים מאותה קטגוריה

↩ חזרה לשלב 2.1

2.3 סטודנט: מעדכן כמות או מסיר פריט מהעגלה במידת הצורך

3 סטודנט: מאשר את העגלה ועובר לתשלום

4 לפי אמצעי התשלום שבחר הסטודנט:

4.1 כרטיס סטודנט

4.1.1 סטודנט: מאשר חיוב של חשבון הסטודנט שלו

4.1.2 המערכת: שולחת בקשת חיוב למערכת החשבונות של האוניברסיטה ומקבלת אישור

4.1.2a אם מערכת החשבונות דוחה את החיוב (יתרה לא מספיקה):

4.1.2a1 המערכת מציגה את סירת הדחייה ומשאירה את העגלה שמורה

ניתוח דרישות

Create, Read, Update, Delete = CRUD · יצירה, קריאה, עדכון, מחיקה

מתי כותבים חריג (extension)?

גם חלופית וגם חריג הם "תלוי במשהו" — אז מה מבדיל ביניהם?

הקו המפריד הוא הצלחה — לא ודאות

השאלה היחידה: האם המערכת הצליחה לעשות את מה שהשחקן ביקש?

חלופית (alt) — כן הצליחה

השחקן בחר מסלול תקין אחר. כל ענף = מערכת עובדת.

"שילם בכרטיס סטודנט" · "שילם באשראי"
שניהם — ההזמנה שולמה.

חריג (extension) — לא הצליחה

המערכת לא יכלה לבצע. קלט שגוי · נדחה · לא נמצא · אזל ·
timeout.

"הסולק דחה את החיוב" · "קוד האיסוף שגוי"
תלוי בצעד שנכשל — לא ברשימה נפרדת.

כל חריג חייב להיגמר — לרשום את הבעיה זה חצי עבודה

↪ חזרה לשלב 2.1
השליטה חוזרת לצעד מסוים

⊖ המקרה מסתיים ללא הצלחה
ומציינים באיזה מצב נשארנו

אם אינכם מוצאים אף דרך שתרחיש נכשל — עוד לא חשבתם עליו.

תנאי קדם ותנאי סיום

מה נכון לפני שמתחילים — ומה מובטח אחרי שסיימנו

תנאי קדם (Pre-condition)

מה שהמערכת מניחה שנכון ברגע שהתרחיש מתחיל. אם זה לא מתקיים — התרחיש בכלל לא רץ.
אימות איסוף: ההזמנה מסומנת "מוכנה לאיסוף" · עובד הדלפק מחובר לעמדה.

תנאי סיום (Post-condition)

מה שהמערכת מבטיחה שיהיה נכון אחרי סיום מוצלח. הבטחה — לא משאלה.
אימות איסוף: ההזמנה סומנה "נאספה" ונרשם מועד המסירה · היא הוסרה מרשימת הממתינות.

⚠ תנאי קדם ≠ בדיקה

אם ה-UC בודק את זה ויכול להיכשל על זה — זה צעד + חריג, לא תנאי קדם.

✓ המבחן: אפשר לראות את זה?

מצב שהשתנה · רשומה שנשמרה · הודעה שנשלחה. "הסטודנט מרוצה" — לא נבדק.

💡 כאן חי ה-login

"המשתמש מחובר" הוא תנאי קדם — ולכן login אינו use case בפני עצמו.

⚠ וכשנכשלו?

תנאי הסיום מתאר את מסלול ההצלחה. מה קורה בכישלון — נכתב בחריג (⊖) "העגלה נשמרה, ההזמנה לא נוצרה".

למה אב-טיפוס (prototype)?

מסך גס מגלה תוך דקה מה שרשימת צעדים מסתירה חודש

1. אף אחד לא "קורא" זרימה

תנו למנהל הקפיטריה תשעה צעדים ממוספרים — הוא יאשר. תנו לו מסך — הוא יגיד "רגע, איפה המחיר?"

2. המקום הזול ביותר לטעות

לתקן שדה חסר בסקיצה = קו במחקר. אחרי הפיתוח = ספרינט.

3. מגלה את מה שהזרימה פספסה

כשמציירים את מסך האיסוף מתברר מיד: מה רואים כשהקוד שגוי? המסך שואל את שאלות החריגים בשבילכם.

⚠ אב-טיפוס ≠ עיצוב

קופסאות אפורות. צבעים ופונטים בשלב הזה = ויכוח על ה-איך לפני שסגרנו את ה-מה.

אימות איסוף — סקיצה

עמדת דלפק — אימות איסוף

קוד איסוף

הזמנה #4127

תשלום שולמה

כמות	פריט
1	סלט יווני
2	מיץ תפוזים

דווח על בעיה

סמן כנאספה

...ומה מופיע כאן כשהקוד שגוי? וכשההזמנה עדיין בהכנה?

💡 בפרויקט שלכם

לכל use case שהשחקן שלו הוא אדם — מצרפים סקיצת מסך.

ניתוח דרישות

41

לשונית = פרק בתיעוד

- ## שימו לב ב-Flow of Events

הכלי לא חושב במקומכם

VP נותן את הטופס. מה נכנס לתוכו — זז העבודה שלכם.

Create, Read, Update, Delete = CRUD · יצירה, קריאה, עדכון, מחיקה

מה שהתרשים לא יכול להחזיק

הנחה · משמעות של מילה · למה החלטנו — לכל אחד מהם חייב להיות בית מפורש

מילון מונחים — שם הוא החלטה, לא תווית

המלכודת אינה חוסר עקביות — היא אותה מילה, שתי משמעויות. אצלנו: "עמדה" = עמדת הכנה במטבח (חם/קר)... או עמדת הדלפק? שתי מערכות שונות.

שם התרחיש יהפוך למחלקה, לטבלה ולמסך — שינוי שם מאוחר נוגע בכל השרשרת.

AI "מסתדר" עם זה מצוין 🤖

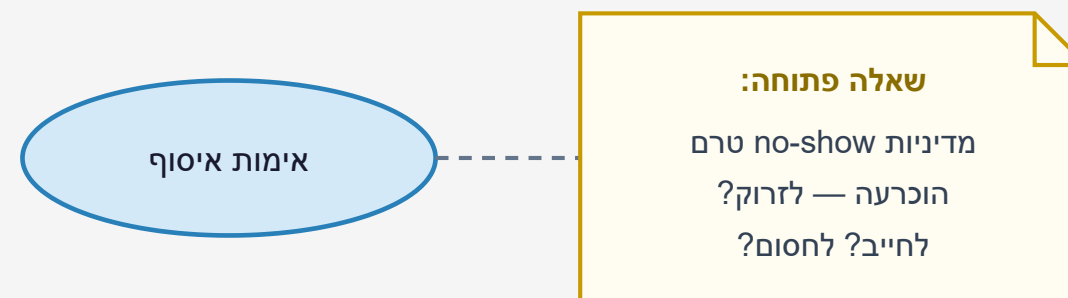
הוא מתמודד "בלי בעיה" עם מילים נרדפות או דומות — ופה בדיוק הסכנה: הוא לא ייעצר לשאול "באיזו עמדה מדובר?" — פשוט יבחר אחת, בשקט.

ניהול שינויים — לא למנוע שינוי, להזיל אותו

שינוי אינו כישלון הניתוח. השאלה היחידה: כמה עולה לגלות מה נשבר? בשביל זה מספרנו — FR-4 השתנתה ← איזה תרחיש מממש אותה ← איזה צעד ← איזה מסך.

שאלה פתוחה שמסומנת ככזו = יושרה. שאלה פתוחה שמתחזה להחלטה = חוב.

הערה (Note) — הנוטציה



מלבן עם פינה מקופלת · קו מקווקו, בלי חץ · נצמדת לכל אלמנט — תרחיש, שחקן, קשר או גבול המערכת.

⚠ להערה אין סמנטיקה

הכלי מתעלם ממנה. לכן היא לא דלת מילוט: מה שאפשר למדל — מדלו. הערה היא למה שבאמת לא ניתן למדל — הנחה, שאלה פתוחה, כלל עסקי.

המודל לא נועד להיות צודק — הוא נועד להפוך טעות לזולה.

ארבעת הקשרים — וזה הכל

Generalization · extend · include · Association

הקשרים בתרשים

Generalization

הורשה (בין שחקנים)

«extend»

התנהגות מותנית, לפעמים

«include»

שימוש חוזר, תמיד מתבצע

Association

קו בין שחקן לתרחיש
(תקשורת)

⚠ מה זה לא קשר

- אין סדר/רצף בין תרחישים (זה לא flowchart)
- אין «precedes»/«uses» (מ-1 UML, מת)
- אין קשר בין שני שחקנים מלבד הורשה

אין קשר חמישי. אם צריך לתאר סדר פעולות — זה ה-flow of events, לא התרשים.

Association — הקשר הבסיסי

הקו שמחבר שחקן לתרחיש

קו פשוט בין שחקן לתרחיש = תקשורת. זהו הקשר הבסיסי ביותר.

דוגמה מהקמפוס



שני שחקנים לאותו תרחיש — כל אחד פותח את התור בעצמו, מתי שהוא צריך. אין ביניהם היררכיה.

הסימון

- קו מלא — בלי חץ
- כל תרחיש מחובר לפחות לשחקן אחד
- שחקן = מי שיוזם. אין "ראשי" ואין "משני" — יש שחקנים
- לתרחיש יכולים להיות כמה שחקנים — כל אחד יכול ליזום אותו בעצמו

⚠ מה שאנחנו קוראים לו — אינו שחקן

שער תשלום שהמערכת פונה אליו הוא קריאת API — עיצוב, לא תרשים. המבחן: הוא מפעיל אותנו, או אנחנו אותו? אילו היה מתריע לנו מיוזמתו — היה שחקן.

ניתוח דרישות

«include» — שימוש חוזר

מחלצים תת-התנהגות משותפת

- כשכמה תרחישים חולקים תת-התנהגות — מחלצים אותה לתרחיש משלה
- התרחיש המשותף תמיד מתבצע (mandatory) — ללא תנאי
- החץ: חץ מקווקו מהתרחיש הבסיסי אל המשותף, עם המילה «include»

בדוגמה שלנו:

הזמנת אוכל «include» בדיקת עגלה • תשלום על הזמנה «include» בדיקת עגלה

הבינה מאתרת שימוש חוזר

Claude קורא את כל התרחישים, מזהה תת-התנהגות שחוזרת בכמה מהם (למשל בדיקת עגלה — גם בהזמנה וגם בתשלום) ומציע לחלץ אותה לתרחיש «include» אחד.

בדקו את הכיוון והתנאי

החץ מצביע מהתרחיש הבסיסי אל המשותף — לא הפוך. ושימו לב: «include» תמיד רץ. אם ההתנהגות מתרחשת רק לפעמים — זה כבר לא include.

«include» — איך זה נראה באמת

התרשים אומר שיש שימוש חוזר — הזרימה אומרת איפה הוא נקרא

זרימה: הזמנת אוכל

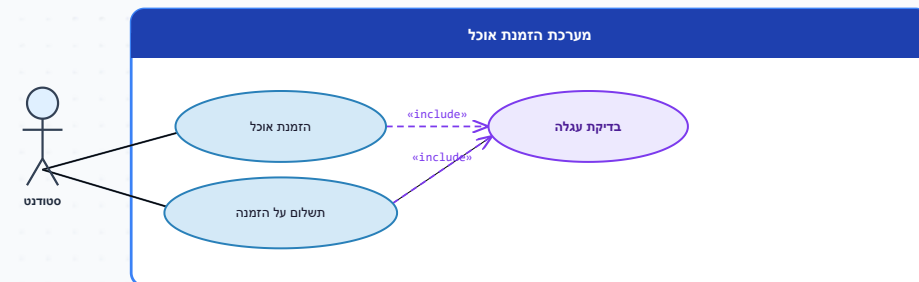
1. סטודנט: פותח את העגלה ובוחר "בצע הזמנה"
2. «include» בדיקת עגלה
3. סטודנט: בוחר זמן איסוף
4. המערכת: יוצרת הזמנה ומציגה סיכום

זרימה: תשלום על הזמנה

1. סטודנט: בוחר הזמנה ולוחץ "שלם"
2. «include» בדיקת עגלה
3. סטודנט: מזין את פרטי התשלום
4. המערכת: מעדכנת את ההזמנה ל"שולמה"

אותה שורה — בשתי הזרימות

כל חץ מקווקו בתרשים = שורה אחת בזרימה. שם, ורק שם, ה-«include» באמת נקרא.



👉 לחצו על בדיקת עגלה — אין לה שחקן, אין לה מסך, והצעדים שלה לא יודעים מי קרא להם.

שני בסיסים • בלי שחקן משלה • רצה תמיד — רק אז זה «include».

«extend» - התנהגות מותנית

מוסיפים התנהגות אופציונלית בתנאי

- תרחיש מרחיב מוסיף התנהגות אופציונלית לתרחיש בסיסי
- בנקודת הרחבה (extension point) ובתנאי (condition)
- החץ הפוך מ-include: מהמרחיב אל הבסיסי. קורה רק לפעמים

נקודת הרחבה ותנאי

- נקודת הרחבה (extension point) — היכן בבסיסי נכנס המרחיב
- תנאי (condition) — מתי בכלל הוא קורה

הסימון: «extend»

Extension - - - ▷ Base

חץ מקווקו עם ראש פתוח, מהתרחיש המרחיב אל הבסיסי — ומעליו התווית «extend».

בדוגמה שלנו:

הזמנה מראש «extend» הזמנת אוכל [תנאי: איסוף עתידי]

⚠ **כלל אצבע — בלבול נפוץ (גם של AI)**

include = תמיד · extend = בתנאי. בלבול ביניהם הוא טעות נפוצה (גם של AI): מודל יסמן לעיתים התנהגות אופציונלית כ-include חובה, או הפוך. בדקו את כיוון החץ ואת התנאי.

«extend» — ולמה לא פשוט תרחיש?

אותו מה, איך אחר — והשאלה האמיתית היא מי צריך לראות את זה

למה לא תרחיש?

תרחיש מסתתר בתוך התיעוד. «extend» נמצא בתרשים — ורק שם הלקוח יכול להצביע ולשאול "איפה זה אצלכם?"

1. הזמנה מראש — זה שאפשר להתווכח עליו

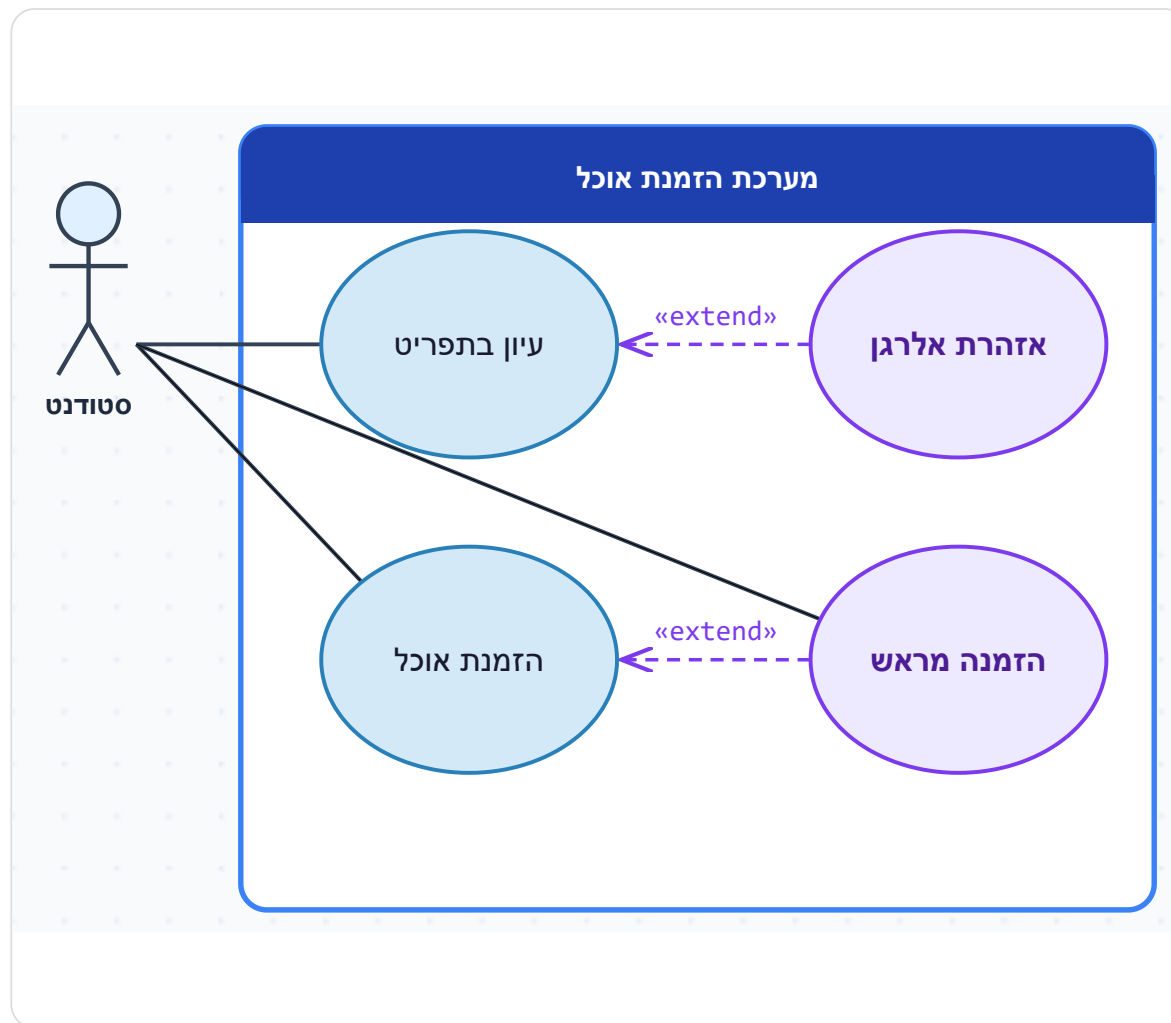
יכלה להיות תרחיש נסתר ("אותה מטרה, מסלול אחר"). בחרנו להראות — כי המנהל מודאג מ-no-show, ועכשיו הוא רואה את זה ושואל: "ומה אם הוא לא בא?"
נקודת הרחבה: קביעת זמן איסוף · תנאי: איסוף עתידי · נדיר: "לפעמים"

2. אזהרת אלרגן — זה שאין עליו ויכוח

לא תרחיש: אותו מסלול בדיוק, אותה מטרה — רק התנהגות נוספת לתת-קבוצה נדירה. ובטיחותי, ולכן הלקוח חייב לראות שהמערכת תומכת בזה.
נקודת הרחבה: הצגת פרטי הפריט · תנאי: מוגדר פרופיל אלרגיה

⚠️ ההבדל מ-«include»

את ה-«include» קוראים — יש לו שורה בזרימה. את ה-«extend» אף אחד לא קורא: הבסיס מציין נקודת הרחבה, וההרחבה נכנסת שם — רק אם התנאי מתקיים.



👉 לחצו על ההרחבות. שימו לב: שני הבסיסים רצים מצוין בלעדית.

אותו מה, איך אחר · נדיר · ורוצים שיראו אותו — רק אז «extend».

Generalization — הורשה

בין שחקנים — שחקן כללי שממנו יורשים שחקנים מתמחים

הורשת שחקן (Actor)

צוות קפיטריה

צפייה בתור הכנה **המשותף**



מנהל קפיטריה

+ ניהול תפריט ומלאי

+ צפייה בדוח מכירות

עובד מטבח

+ סימון הזמנה כמוכנה

עובד דלפק איסוף — לא בהיררכיה. הוא לא צופה בתור ההכנה, אז אין לו חלק במשותף (הוא עושה רק אימות איסוף). ילד שלא עושה את המשותף — לא ילד.

המשולש החלול מצביע אל ההורה — אותו סימון כמו ב-Class Diagram, כאן בין שחקנים.

מתי זה נכון? — שני תנאים, שניהם

- 1. המנהל צופה בתור ההכנה בתור מנהל — זה חלק מהתפקיד שלו, **לא כממלא מקום**
- 2. ויש לו גם תרחישים משלו — ניהול תפריט, דוח מכירות. אחרת הוא פשוט "צוות קפיטריה", ולא צריך שחקן נפרד בכלל

נפל תנאי 1 ← ממלא מקום (למטה). נפל תנאי 2 ← הוא פשוט השחקן הכללי. השחקן הכללי מחזיק את המשותף; כל מתמחה יורש אותו ומוסיף את שלו — וחוסך קווי association חוזרים.

שחקן = תפקיד, לא אדם 🎭

המנהל עומד בדלפק כי עובד הדלפק יצא להפסקה. באותו רגע הוא לא פועל בתור מנהל — הוא ממלא מקום: אותו אדם, תפקיד אחר. זו לא הורשה — הקשר שייר לעובד דלפק, והאדם פשוט חובש שני כובעים בזמנים שונים. השאלה אף פעם אינה "מי האדם?" אלא "באיזה תפקיד הוא פועל עכשיו?"

⚠ בקורס זה: הורשה רק בין שחקנים

UML מתיר generalization גם בין תרחישים — אנחנו לא משתמשים בזה בקורס (תיתקלו בזה, ולא — זו לא טעות). לשיתוף התנהגות בין תרחישים: «include» (תמיד) או «extend» (בתנאי).

הורשה – איך זה נראה באמת

קו אחד על ההורה – ושני התפקידים יורשים אותו

מה לחפש – מה שאיננו שם 🔍

אין קו מעובד מטבח או ממנהל אל "צפייה בתור הכנה". הם יורשים אותה מצוות קפיטריה. ההיעדר הזה – הוא הסימון שעובד.

שני התנאים מתקיימים – ולכן זו הורשה ✅

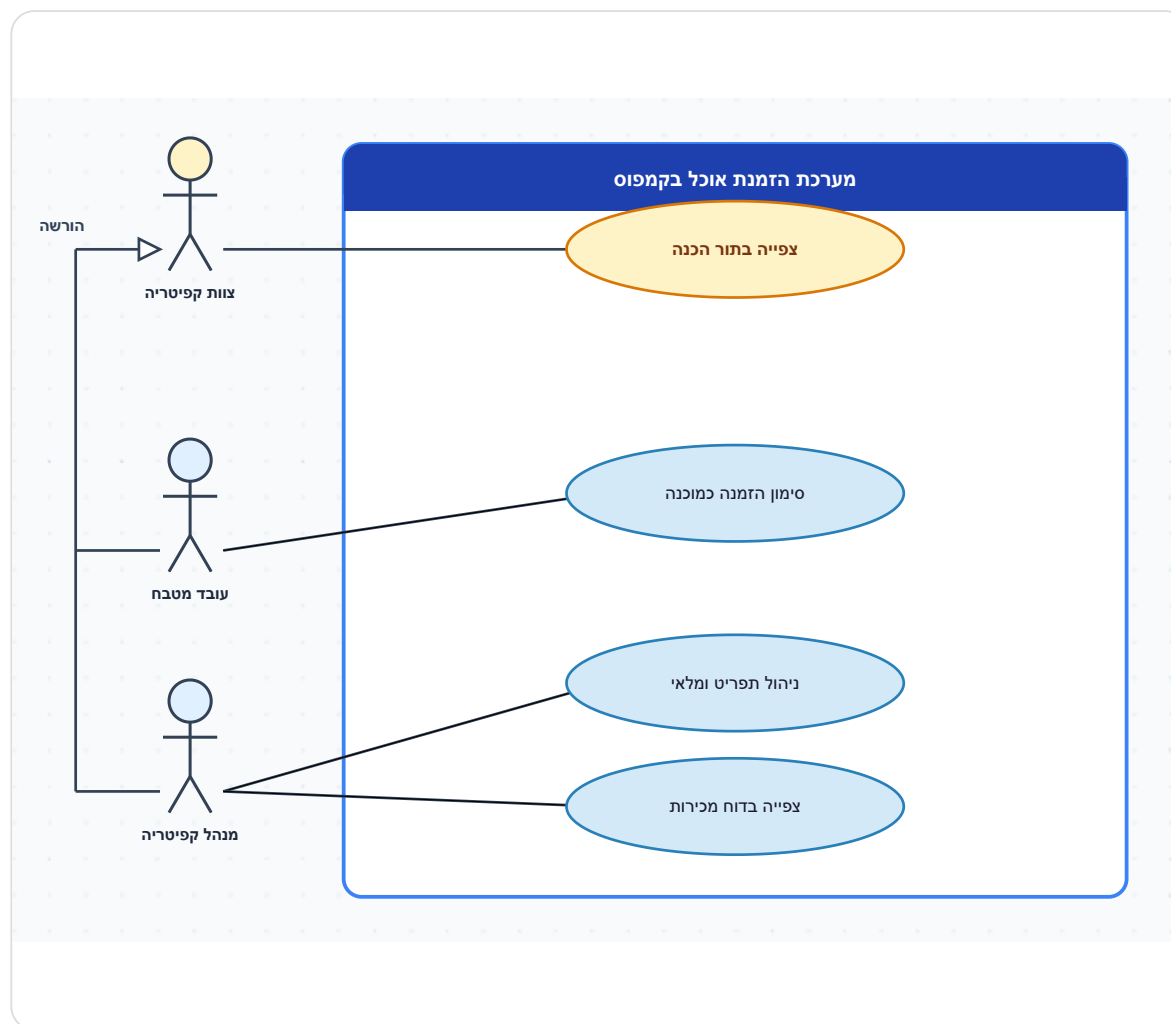
1. שניהם צופים בתור ההכנה בתור עצמם – זה חלק מהתפקיד, לא מילוי מקום.
2. ולכל אחד יש גם משלו: מטבח מסמן הזמנה כמוכנה; המנהל מנהל תפריט ורואה דוח. נפל תנאי 2? אין ילד – הוא פשוט "צוות קפיטריה".

מי לא בהיררכיה – ולמה 📌

עובד דלפק איסוף. הוא לא צופה בתור ההכנה – אין לו חלק במשותף, אז אין מה לרשת. ילד שלא עושה את המשותף – לא ילד. ומה עם המנהל שמחליף אותו בהפסקה? זה מילוי מקום – ברגע שהוא הוא עובד דלפק.

⚠️ ורגע – בתרשים הגדול יש שני קווים!

נכון, ושניהם תקינים. עם תרחיש משותף אחד – שני קווים פשוטים יותר. ההורשה משתלמת כשהמשותף גדל. זו החלטה, לא כלל.



👉 לחצו על השחקנים, על התרחישים – ועל קו ההורשה עצמו.

ההורה מחזיק את המשותף · כל ילד מוסיף את שלו · מי שלא חולק – לא ילד.

איזה קשר — אם בכלל?

לחצו על מקרה — ותקבלו שתי דרכים למדל אותו. איזו נכונה יותר?

4  מנהל ועובד מטבח — שניהם צופים בתור ההכנה, ולכל אחד יש גם משלו 	3  התחברות לפני שמזמינים 	2  הזמנה מראש לשעת איסוף עתידית — לפעמים 	1  בדיקת עגלה — נדרשת גם בהזמנת אוכל וגם בתשלום, בכל פעם 
8  הסטודנט תמיד מעיין בתפריט לפני שהוא מזמין 	7  פריט אזל בזמן ההזמנה — המערכת מציעה חלופה 	6  הדפסת מדבקה אחרי סימון "מוכנה" — בכל פעם 	5  תשלום — בכרטיס סטודנט או באשראי 

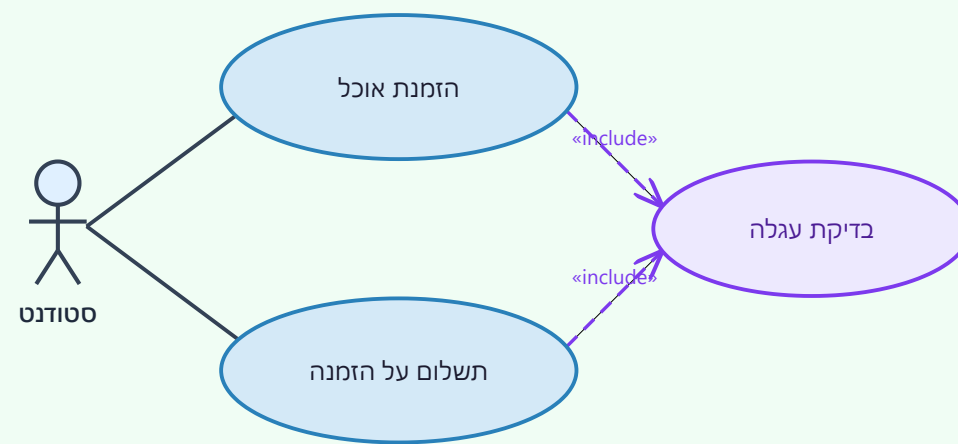
ניתוח דרישות

רמז: ברירת המחדל היא אפס «include» ואפס «extend».

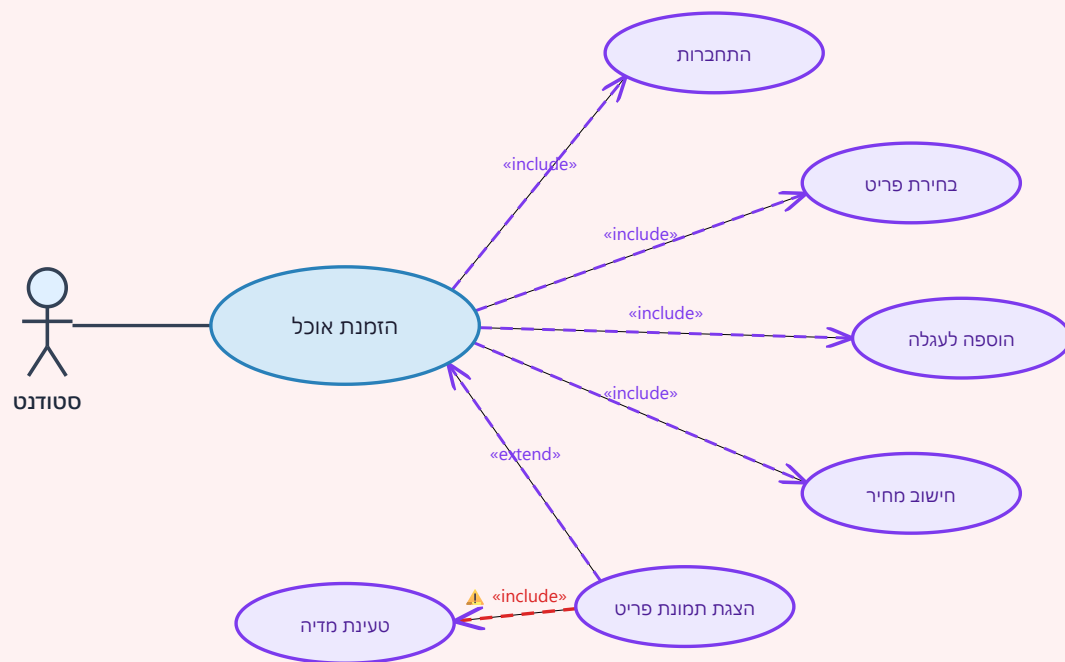
KISS — פירוק טוב מול פירוק-יתר

אותו תרחיש: כמה תרחישים ברורים — או עשרות זעירים?

✓ **פירוק טוב** — תרחישים שמניבים ערך, «include» אחד מוצדק



✗ **פירוק-יתר** — כל צעד זעיר הפך לתרחיש (אנטי-תבנית)



1 • אין

«extend» → «include»

לא משרשרים הרחבה לתוך הכללה (הקו האדום).



2 • «include» → «extend»

קורה, אבל
נדיר מאוד
— עצרו ובדקו.

3 • «extend» במשורה

ברירת המחדל
אפס
. רוב הפריטים — צעד בזרימה.

פשוט מנצח מפורט — התרשים מתקשר היקף וערך, לא מפרק טכני.

• ניתוח דרישות

Keep It Simple, Stupid = KISS · שמור על פשטות

רמת פירוט ו-FAQ

כמה זה יותר מדי — וכמה מעט מדי

לא גבוה מדי, ולא נמוך מדי

לא כל לחיצה היא תרחיש, ולא הכול תרחיש אחד ענק. תרחיש = יחידת ערך אחת לשחקן.

לחבר שחקן ל-«extend»?

כן. להרחבה יכול להיות שחקן (הוא בוחר בה) — או לא (המערכת מפעילה אותה לפי תנאי).

לחבר שחקן ל-«include»?

בדרך כלל לא — ליעד «include» טהור אין שחקן. חוקית מותר, אבל אם יש לו שחקן — כנראה שהוא לא באמת «include».

האם login הוא use case?

לא — זה תנאי קדם, לא תרחיש. וגם לא יעד «include»: אי אפשר לכלול משהו שאינו תרחיש.

כמה תרחישים צריך?

תלוי בהיקף — אין מספר קסם. מודדים לפי מטרות הערך של השחקנים, לא לפי מסכים.

קשר «extend» מול "הרחבות" בטבלת ה-Flow of Events?

קשר בתרשים = תהליך שונה משמעותית. חריג בתוך הזרימה = משהו פשוט כמו if...else.

שחקן "זמן" נפרד לכל פעולה מתוזמנת?

לא — שחקן 🕒 אחד, וכמה חיצים ממנו בתדירויות שונות (יומי, שעתי, בתום חלון...).

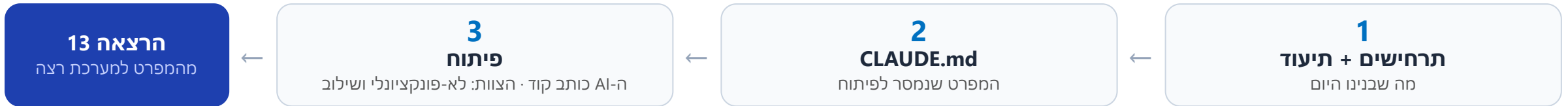
🤖 "כשוט אשאל את ה-AI?"

כן — אבל אתם מאמתים גם את רמת הפירוט וגם את הדוקטרינה (login זה לא תרחיש, extend במשורה...).

ניתוח דרישות

מניתוח לפיתוח – ותפקיד האנליסט

המפרט שבניתם עובר הלאה: ה-AI כותב את הקוד – ואתם מחליטים ומאתגרים



💡 המפרט הוא הפרומפט

ההבנה שלכם היא התקרה של מה ש-AI יפיק: מפרט חד ← קוד טוב; עמימות ← אותו באג, בביטחון ובקנה מידה. מודל תרחישי השימוש הוא בדיוק מפרט כזה.

פעם

רוב הזמן הלך על ייצור התוצרים – לצייר תרשימים, לכתוב תיעוד, לעצב טבלאות.

היום

הערך עובר להחלטה ולאימות: לשאול את השאלות הנכונות, להגדיר היקף, ולאתגר את מה ש-AI מייצר.

שיקול דעת, ידע בתחום, ויכולת לאתגר – המיומנות הנדירה שנשארת.

github.com/dcodish/SAD-sample-project — פרויקט הדוגמה להרצאה 13

ניתוח דרישות

תרגול: מהמסמך לתרשים — ואז ביקורת

מזינים מסמך אפיון מלא ל-AI, מקבלים תרשים — ואתם השופטים



אתם מבקרים

מה חסר? מה הומצא? מה הונח?



ה-AI מייצר תרשים

שחקנים, תרחישים, קשרים — תוך שניות



מסמך אפיון מלא

ארגון · מצב קיים · ראיונות · 100+ דרישות

P חניון 'כיכר המרכז' **בבית**

136 דרישות **FR · NFR · DR · BR** שער תשלום = API תזמון

חניון עירוני בתשלום: כניסה, יציאה, מנויים, איכפה. אותו תהליך בדיוק — זו מטלת הבית (השקף האחרון).

practice/parking.html

ספרייה עירונית 'אשכול' **בכיתה**

152 דרישות **FR · NFR · DR · BR** הורשת שחקנים תזמון

רשת של 3 סניפים: השאלות, הזמנות, קנסות, רכש מספקים. נריץ יחד בכיתה ונפרק את התרשים על המסך.

practice/library.html

⚠ המיומנות שמתרגלים

השאלה איננה "מה ה-AI ייצר" אלא "מה הוא פספס, המציא, או הניח" — שחקן חסר, «include» מומצא, מערכת חיצונית שסומנה כשחקן במקום קריאת API, גבול שגוי.

ניתוח דרישות

אותו מסמך, אותו AI — ושתי כיתות יקבלו ביקורת שונה. הביקורת היא התוצר.

Non-Functional Requirement = NFR · דרישה לא-פונקציונלית · Functional Requirement = FR · דרישה פונקציונלית · Artificial Intelligence = AI · בינה מלאכותית

מה למדנו — ולאן ממשיכים

- UML רלוונטי — ודווקא בעידן AI הסמנטיקה חשובה יותר
- Use Case = שחקנים + תרחישים + גבול מערכת
- ארבעה קשרים: generalization, extend, include, association
- התיעוד (Flow of Events) הוא התוכן; התרשים הוא המפה
- AI מאיץ כל שלב — ואתם מאתגרים ומאמתים

הבא בתור: תרשים מחלקות (Class Diagram) — הרצאה 6-7
נעבור מ'מה המערכת עושה' ל'ממה היא בנויה'.

אותו תהליך שעשינו בכיתה — על הדוגמה שלא הרצנו

קחו את מסמך האפיון של החניון 'כיכר המרכז'

מסמך מלא: תיאור הארגון, מצב קיים, ראיונות, ו-136 דרישות מפורטות (FR/NFR/DR/BR). הקובץ: [lecture-04-05/practice/parking.html](https://www.dropbox.com/s/04-05/practice/parking.html?dl=1)

1

הזינו ל-AI

תנו את המסמך ל-Claude ובקשו תרשים flow of + use case events לשניים-שלושה תרחישים מרכזיים.

2

בקרו לפי הדוקטרינה

מי באמת שחקן ומי API שאנחנו קוראים לו? יש «include» מומצא (למשל login)? «extend» שהוא בעצם חריג? גבול נכון?

3

תקנו ונמקו

הציעו תרשים מתוקן, והסבירו כל שינוי — מה ה-AI פספס, המציא, או הניח, ולמה התיקון שלכם נכון.

מה מגישים

- הפלט המקורי של ה-AI (התרשים והתיעוד שקיבלתם) + הפרומפט ששלחתם.
- טבלת ביקורת: לכל ליקוי — מה הבעיה, לאיזה כלל היא נוגעת, והתיקון.
- התרשים המתוקן שלכם (מהכלי או ביד), ופסקה קצרה: מה ה-AI דווקא עשה טוב.

לא מדרגים כמה יפה ה-AI צייר — מדרגים כמה חד הייתה הביקורת שלכם.

Non-Functional Requirement = NFR · דרישה לא-פונקציונלית · Functional Requirement = FR · דרישה פונקציונלית · Artificial Intelligence = AI · בינה מלאכותית

תודה!

הבא בתור — תרשים מחלקות (Class Diagram): ממה המערכת בנויה

ניתוח ועיצוב מערכות מידע · הרצאות 4-5 — תרשים תרחישי שימוש Use Case